Python Programming – Day 4

Class Notes & Summary

Opening Activity – Student Speaking Session (Start of Class)

The instructor intentionally opened the class with a short student speaking activity before diving into technical content. Two students were asked to come up and speak freely — about a movie/series they liked, or any topic of their choice.

What Happened

- Student 1 spoke about a web series called Frozen and a movie called The Road. She described the storyline — a road with repeated accidents, superstitious beliefs around it, the government ignoring it, and a woman who investigated and uncovered a story of theft and murder.
- After she spoke, the instructor asked the rest of the class to say one keyword they remembered from her talk. Students called out: moving, jewelry, again and again, curiosity, superstitious, web series, kill.
- The instructor then summarized the communication test: if you convey your message and the listener understands → 50 marks. If the listener can summarize back → another 50 marks. That equals 100. You are good at communication.

> 🎤 **Speaking Activity Note:** *This opening activity was fully intentional — the goal was to build student confidence in speaking in front of a group. This is an extremely valuable habit for tech professionals. Communication is a core skill in any software job — from explaining your code in a standup meeting, to client presentations, to debugging with teammates. Here are a few suggestions to make this activity even more effective in future sessions:1. Give the speaker a specific time limit (e.g., exactly 2 minutes) so they practice being concise, which is a real-world skill.2. Ask the audience to give one piece of positive feedback and one improvement tip — this trains both the speaker and the listeners.3. Gradually increase the difficulty: start with personal topics (movies, hobbies), then move toward technical topics (explain what a variable is, explain an API call in simple words). This bridges communication practice directly into technical communication.*

Closing Activity – Student Speaking Session (End of Class)

At the end of class, the instructor invited students to come up and speak again for about 5 minutes. One student came forward and shared about her engineering background — she started as a biology student, had no idea what Python or Java was, learned slowly through college, did small projects in third year, completed internships, and is now in her final semester.

Another student was invited but was shy and found it difficult to start. The instructor encouraged them gently, asking about their family, college, or favorite movie — anything at all.

> 🎤 **Speaking Activity Note:** *The shy student moment is actually a great learning opportunity. Being uncomfortable speaking in public is very common, especially in technical fields where introverts thrive. The key message to give students: nobody starts confident — confidence is a skill you build by doing it repeatedly, even when it's uncomfortable. The engineering student's story is a great example: she started with zero programming knowledge (biology background) and is now learning Python professionally. That is exactly how growth works. You can reinforce this with a simple message: 'Every time you speak up in this class, even if it is wrong, you are building a skill that will help you in your career more than any line of code.'*

1. Recap – API Call via Browser (from Day 3)

The class started by revisiting Day 3's homework. The instructor showed the GitHub API endpoint and recapped what was done:

- Method 1 (done in Day 3): Open the GitHub API URL in a browser. The browser is the client. It sends a GET request to the GitHub server and gets back a JSON response body.
- The response body contains your GitHub profile info: id, login, name, type, company, location, twitter_username, followers, public_repos, etc.
- Task from Day 3 was to pick 5 keys from the response and print them using Python.

GitHub API Endpoint

```
https://api.github.com/users/{your_github_username}
```

No login credentials needed — this is a publicly accessible endpoint.

> 💡 **Instructor Note:** *URL and API endpoint mean the same thing. Both terms refer to the address you call to get a response from a server. Get comfortable using both terms interchangeably — you'll hear both in professional settings.*

2. API Call via Python – requests Library (Continued)

Method 2: Making the same API call using Python code instead of a browser.

The Code

```
import requests

url = "https://api.github.com/users/your_username"
```

```
response = requests.get(url)

data = response.json()    # Convert JSON to Python dictionary

print(data.get("id"))
print(data.get("type"))
print(data.get("location"))
print(data.get("twitter_username"))
print(data.get("followers"))
```

JSON → Dictionary (Important Reminder)

Python does not understand JSON natively. The response body comes as JSON. You MUST convert it to a Python dictionary using response.json() before you can access keys.

response	`The raw response object from requests.get()`
response.status_code	`Returns the HTTP status number (e.g., 200)`
response.json()	`Converts JSON response body to Python dictionary`
data.get("key")	`Safely retrieves a value from the dictionary by key`

Common Error: Missing Schema

If you forget to include https:// at the start of the URL, you will get a 'MissingSchema' error. The URL must always start with http:// or https://.

```
# Wrong:
url = "api.github.com/users/username"   # MissingSchema error!
# Correct:
url = "https://api.github.com/users/username"
```

> 💡 **Instructor Note:** *Always double-check your URL when you get a MissingSchema error. It almost always means you forgot the https:// prefix. Also make sure there are no extra spaces or typos in your GitHub username.*

3. Operators in Python

The session then moved into a core Python concept: operators. An operator is a symbol that performs a specific operation on values or variables.

Types of operators discussed in Day 4:

- Arithmetic Operators
- Comparison (Relational) Operators
- Assignment Operator
- Logical Operators

3.1 Arithmetic Operators

Arithmetic operators perform mathematical calculations. They work on integer and float data types.

+ (Addition)	`a + b → 10 + 20 = 30`
- (Subtraction)	`a - b → 20 - 10 = 10`
*** (Multiplication)**	`a * b → 10 * 3 = 30`
/ (Division)	`a / b → 10 / 3 = 3.333... (exact answer, float)`
// (Floor Division)	`a // b → 10 // 3 = 3 (removes decimal, rounds DOWN)`
% (Modulus)	`a % b → 10 % 3 = 1 (gives REMAINDER only)`
**** (Power/Exponent)**	`a ** b → 10 ** 2 = 100 (10 raised to power 2)`

Floor Division vs Regular Division – Explained

A key concept that was demonstrated live:

- / gives the exact result including the decimal: 10 / 3 = 3.333...
- // gives only the whole number part, always rounded DOWN (floor): 10 // 3 = 3
- Floor = always take the lower value. Even 3.999 becomes 3 with floor division.
- Ceiling (not discussed deeply) = always take the upper value. 3.1 would become 4.

> 💡 **Instructor Note:** *Floor division is not just a Python thing — it comes from mathematics. The 'floor' of a number is the largest integer less than or equal to that number. This becomes very useful in real programs when you need whole-number calculations, like dividing items into groups or calculating pages.*

Modulus – Explained with a Real Use Case

Modulus (%) gives the remainder after division. It is one of the most useful operators once you understand when to apply it.

The instructor demonstrated a pattern live: when you keep increasing a variable and do a % 10:

```
0 % 10 = 0
```

```
1 % 10 = 1
2 % 10 = 2
...
9 % 10 = 9
10 % 10 = 0   ← resets to 0
11 % 10 = 1   ← starts again
20 % 10 = 0   ← resets again
```

This is a cycling/looping pattern. When you want behavior that repeats in a fixed cycle, modulus is the tool.

> 💡 **Instructor Note:** *Classic real-world uses of modulus: checking if a number is even or odd (n % 2 == 0 means even), building a circular queue, rotating through a fixed list of items, creating a progress bar that wraps around, or checking divisibility. You'll use % more than you expect once you start building real applications.*

Important: + With Strings vs Numbers

When you use + between two strings, it is concatenation (joining), not addition.

```
a = "school"
b = "college"
c = a + b   →   "schoolcollege"   (concatenation, NOT 30)
```

Arithmetic operations must be done on int or float types only. If your variables are strings, + will concatenate them instead of adding.

3.2 Comparison (Relational) Operators

Comparison operators compare two values. The result is always a boolean: True or False.

== (Equal to)	`a == b → 10 == 20 → False`
!= (Not equal to)	`a != b → 10 != 20 → True`
> (Greater than)	`a > b → 10 > 20 → False`
< (Less than)	`a < b → 10 < 20 → True`
>= (Greater or equal)	`a >= b → 10 >= 10 → True (equal part triggers)`
<= (Less or equal)	`a <= b → 10 <= 20 → True`

Important: = vs ==

This is one of the most common mistakes beginners make:

= (one equals)	Assignment operator. Assigns a value to a variable. Example: a = 10 puts the value 10 into a.

== (double equals)	Comparison operator. Checks whether two values are equal. Returns True or False. Example: a == 10 checks if a holds 10.

```
a = 10      # Assignment: put 10 into a
a == 10     # Comparison: is a equal to 10? → True
a == 20     # Comparison: is a equal to 20? → False
```

> 💡 **Instructor Note:** *Accidentally using = instead of == inside an if condition is one of the most common bugs in all programming. Remember: one equals assigns, double equals compares. When checking conditions, always use ==.*

Edge Case: >= and <=

The >= operator checks two things: is it greater? OR is it equal? If either one is true, the result is True. The instructor demonstrated:

```
a = 10, b = 10
a >= b   →   is 10 > 10? No (False).   is 10 == 10? Yes (True).
         →   False OR True = True   →   Overall result: True
```

This is internally like an OR operation — at least one condition needs to be satisfied.

Can You Compare Strings?

Yes — Python can compare strings. The comparison is done alphabetically based on Unicode/ASCII values. This was briefly introduced and will be explored more in future classes.

3.3 Assignment Operator

Assignment operator (=) assigns a value to a variable. The right side provides the value; the left side is the variable that receives it.

```
a = 10      # a gets the value 10
b = 20      # b gets the value 20
a = b       # now a gets whatever value b has → a becomes 20
```

4. Logical Operators

Logical operators combine multiple conditions together. They are used inside if statements when you need to check more than one condition at once.

and	Both conditions must be True for the result to be True.
or	At least one condition must be True for the result to be True.
not	Reverses the boolean value. True becomes False, False becomes True.

and – Both Must Be True

```
if username == "admin" and password == "123":
    print("Login successful")
```

With and: if the first condition is False, Python does NOT even check the second condition — it already knows the result will be False. This is called short-circuit evaluation.

or – At Least One Must Be True

```
if username == "admin" or username == "superuser":
    print("Access granted")
```

With or: even if one condition is False, Python checks the other. As long as one is True, the block executes.

> 💡 **Instructor Note:** *The 'and' short-circuit behaviour is important. In a login check like if username == 'admin' and password == '123', if the username is wrong, Python won't even check the password. This is both efficient and logical — why check the second thing if the first already failed?*

5. Conditional Statements – if / elif / else

Conditional statements let your program make decisions. Based on whether a condition is True or False, different blocks of code get executed.

Structure

```
if condition1:
```

```
    # runs if condition1 is True
elif condition2:
    # runs if condition1 is False AND condition2 is True
else:
    # runs if ALL above conditions are False (fallback)
```

Python uses 'elif', not 'else if'

In most other languages (C, Java, JavaScript), the keyword is else if. In Python, it is shortened to elif. This is a syntax rule — writing else if in Python will cause an error.

Key Rules to Remember

- Only ONE block executes — whichever condition matches first. After that, the rest are skipped.
- else has no condition. It is the fallback — everything that doesn't match any if or elif lands here.
- elif can have a condition. else cannot.
- You can have multiple elif blocks between one if and one else.

Real-World Example – Login System

The instructor built a login example step by step using all four possibilities:

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "123":
    print("Login successful")
elif username == "admin" and password != "123":
    print("Incorrect password")
elif username != "admin" and password == "123":
    print("Incorrect username")
else:
    print("Wrong credentials")
```

The 4 Combinations – Explored Live

With 2 variables that can each be correct or wrong, there are 2 × 2 = 4 possibilities:

Username ✓ Password ✗	`elif: username correct, password wrong → 'Incorrect password'`
Username ✗ Password ✓	`elif: username wrong, password correct → 'Incorrect username'`
Username ✓ Password ✓	`if: both correct → 'Login successful'`
Username ✗ Password ✗	`else: both wrong → 'Wrong credentials'`

> 💡 **Instructor Note:** *The login example is perfect for understanding if/elif/else because it mirrors a real-world scenario everyone understands. The 4-combinations thinking (2 variables × 2 states = 4 possibilities) is a form of logical thinking that helps you write better test cases and catch bugs early. In professional development, this kind of thinking is called 'boundary testing' or 'test case design'.*

How Execution Flows

- Python checks the if condition first.
- If it is True — that block executes. The elif and else are completely skipped.
- If it is False — Python moves to the first elif and checks that.
- If the elif is True — that block executes. Remaining elif and else are skipped.
- If all if and elif conditions are False — the else block executes (always).
- Only ONE block ever executes per if-elif-else chain.

and vs or Inside if Conditions – Recap

Demonstrated specifically with the login example:

```
# Using 'and' — BOTH must match:
if username == "admin" and password == "123":


# Case: username = admin (True), password = 999 (False)
# True AND False = False → if block does NOT execute
# Moves to elif → username correct, password wrong → matches
```

> 💡 **Instructor Note:** *The fact that Python stops evaluating as soon as a False is found in an 'and' chain is a real optimization. In large programs with expensive checks, order your conditions with the quickest/cheapest one first so Python short-circuits early and doesn't waste time on the rest.*

6. Quick Reference – All Operators

+	`Add numbers OR concatenate strings`

-	Subtract
*****	Multiply
/	Divide (exact, returns float)
//	Floor divide (removes decimal, rounds down)
%	Modulus (returns remainder)
******	Power / Exponent (a ** b = a raised to b)
==	Equal to (comparison → True/False)
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
=	Assignment (store a value into a variable)
and	Both conditions must be True
or	At least one condition must be True
not	Reverses the boolean (True→False, False→True)

7. Summary of Key Concepts – Day 4

API Call (Python)	import requests → url = '...' → response = requests.get(url) → data = response.json()
Floor Division	// always rounds DOWN to the nearest whole number. 10 // 3 = 3 (not 4)
Modulus Use Case	% gives remainder. Creates cycling patterns. n % 10 cycles 0→9→0→9...
= vs ==	= assigns a value. == compares two values (returns True/False).
and (both True)	if A and B: — both A and B must be True. If first is False, second is not checked.
or (one True)	if A or B: — either A or B being True is enough to execute the block.

elif (Python only)	In Python, write elif, not else if. This is the correct syntax for chained conditions.

Only one block runs	In an if/elif/else chain, exactly one block executes — whichever condition matches first.

End of Day 4 Notes